C# Arrays

In C#, an array can be of three types:

- Single-dimensional
- Multidimensional
- Jagged Array

Single Dimensional Array

Example: Array Declaration

```
int[] evenNums;  // integer array

string[] cities; // string array
```

Example: Array Declaration & Initialization

```
int[] evenNums = new int[5]{ 2, 4, 6, 8, 10 };

string[] cities = new string[3]{ "Mumbai", "London", "New York" };
```

Accessing Array Elements

Array elements can be accessed using an index. An index is a number associated with each array element, starting with index 0 and ending with array size - 1.

Example: Access Array Elements using Indexes

```
int[] evenNums = new int[5];
evenNums[0] = 2;
evenNums[1] = 4;
//evenNums[6] = 12;  //Throws run-time exception IndexOutOfRange

Console.WriteLine(evenNums[0]);  //prints 2
Console.WriteLine(evenNums[1]);  //prints 4
```

- ## Accessing Array using for Loop

Example: Accessing Array Elements using for Loop

```
int[] evenNums = { 2, 4, 6, 8, 10 };

for(int i = 0; i < evenNums.Length; i++)
    Console.WriteLine(evenNums[i]);

for(int i = 0; i < evenNums.Length; i++)
    evenNums[i] = evenNums[i] + 10;
```

Output?

- ## Accessing Array using foreach Loop

Example: Accessing Array using foreach Loop

```
int[] evenNums = { 2, 4, 6, 8, 10};
string[] cities = { "Mumbai", "London", "New York" };

foreach(int item in evenNums)
    Console.WriteLine(item);

foreach(var city in cities)
    Console.WriteLine(city);
```

- ## Array Methods

All the arrays in C# are derived from an abstract base class

System. Array.

Example: Array Methods

```
int[] nums = new int[5]{ 10, 15, 16, 8, 6 };

nums.Max(); // returns 16
nums.Min(); // returns 6
nums.Sum(); // returns 55
nums.Average(); // returns 55
```

The System.Array class also includes methods for creating, manipulating, searching, and sorting arrays.

Example: Array Methods

```
int[] nums = new int[5]{ 10, 15, 16, 8, 6 };

Array.Sort(nums); // sorts array
Array.Reverse(nums); // sorts array in descending order
Array.ForEach(nums, n => Console.WriteLine(n)); // iterates array
```

- ## Passing Array as Argument

C# Reference types

In **c#**, **Reference Types** will contain a pointer which points to other memory location that holds the data. The **Reference Types** won't store the variable value directly in its memory instead, it will store the memory address of the variable value to indicate where the value is being stored.

An array can be passed as an argument to a method parameter. Arrays are reference types, so the method can change the value of the array elements.

Example: Passing Array as Argument

```
public static void Main(){
    int[] nums = { 1, 2, 3, 4, 5 };

    UpdateArray(nums);

    foreach(var item in nums)
        Console.WriteLine(item);
}

public static void UpdateArray(int[] arr)
{
    for(int i = 0; i < arr.Length; i++)
        arr[i] = arr[i] + 10;
}
```

Output?

C# - Multidimensional Arrays

C# supports multidimensional arrays up to 32 dimensions. The multidimensional array can be declared by adding commas in the square brackets. For example, [,] declares two-dimensional array, [, ,] declares three-dimensional array, [, , ,] declares four-dimensional array, and so on. So, in a multidimensional array, no of commas = No of Dimensions – 1

The following declares multidimensional arrays.

Example: Multidimensional Arrays

```
int[,] arr2d; // two-dimensional array
int[, ,] arr3d; // three-dimensional array
int[, , ,] arr4d ; // four-dimensional array
int[, , , ,] arr5d; // five-dimensional array
```

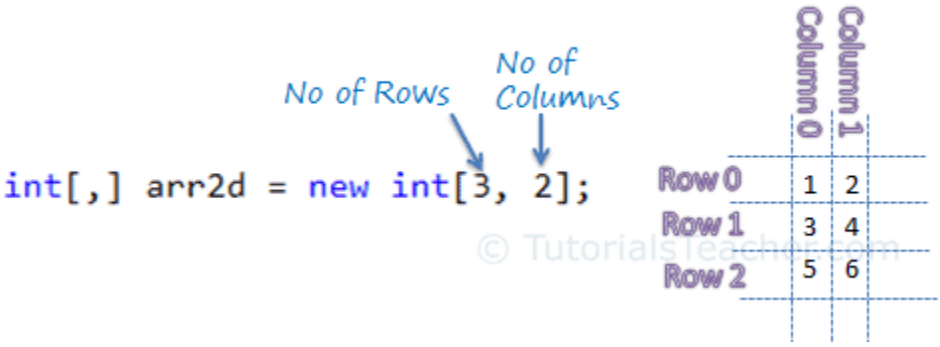
- ## Two Dimensional Array

```
int[,] arr2d = new int[3,2]{
                        {1, 2},
                        {3, 4},
                        {5, 6}
                    };

// or
int[,] arr2d = {
                {1, 2},
                {3, 4},
                {5, 6}
            };
```

In the above example of a two-dimensional array, [3, 2] defines the no of rows and columns. The first rank denotes the no of rows, and the second rank defines no of

columns. The following figure illustrates the two-dimensional array divided into rows and columns.



Example: Access two-dimensional Array

```
int[,] arr2d = new int[3,2]{
                        {1, 2},
                        {3, 4},
                        {5, 6}
                    };

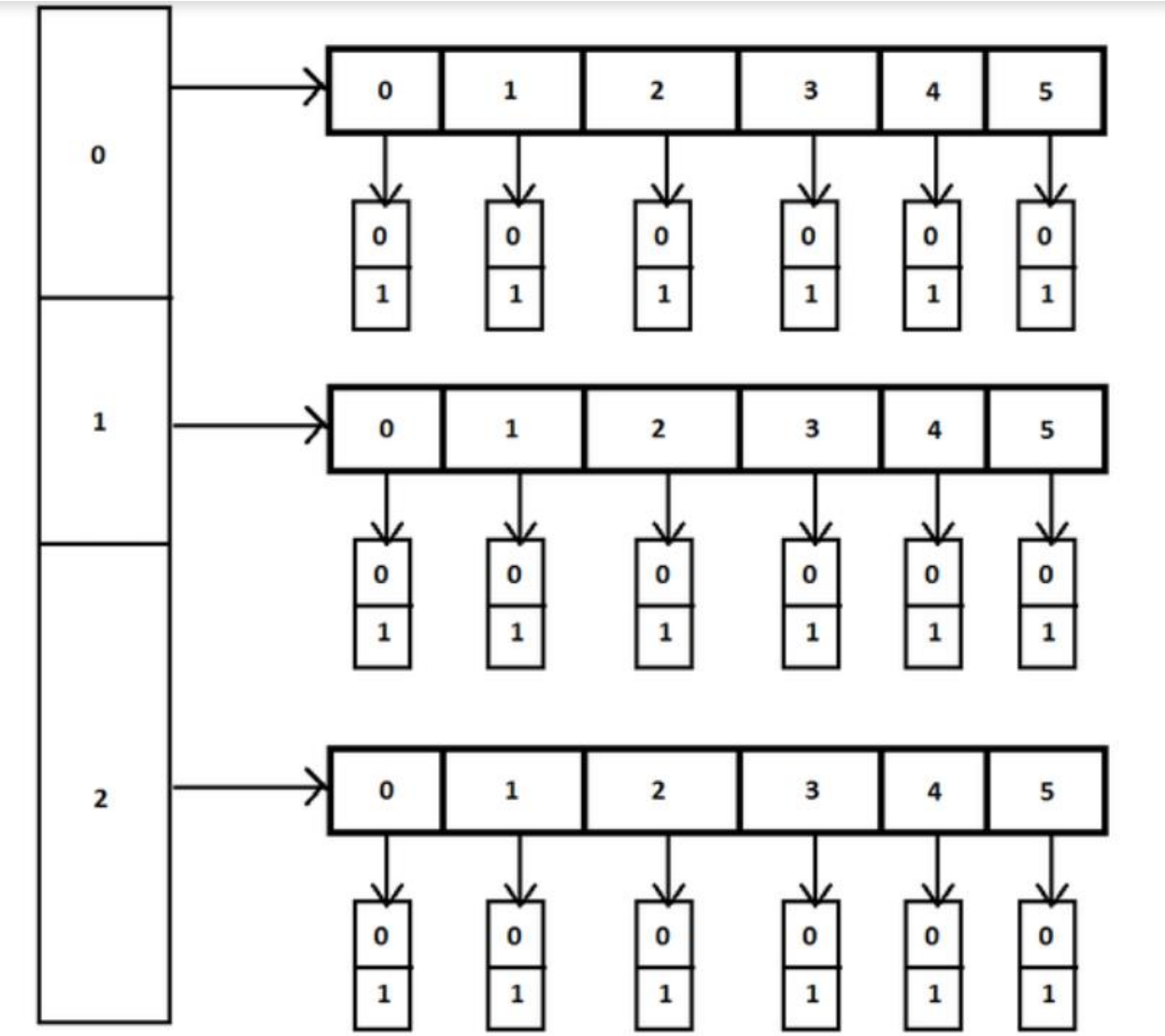
arr2d[0, 0]; //returns
arr2d[0, 1]; //returns
arr2d[1, 0]; //returns
arr2d[1, 1]; //returns
arr2d[2, 0]; //returns
arr2d[2, 1]; //returns
//arr2d[3, 0]; //return ?
```

Output?

In the above example, the value of a two-dimensional array can be accessed by index no of row and column as [row index, column index]. So, [0, 0] returns the value of the first row and first column and [1, 1] returns the value from the second row and second column.

- **Three Dimensional Array**

  Array[3][2][6]



Example: Three-dimensional Array

```
int[, ,] arr3d1 = new int[1, 2, 2]{
            { { 1, 2}, { 3, 4} }  // 1dimension 2 rows 2 cols of a 2D array
        };

int[, ,] arr3d2 = new int[2, 2, 2]{    //2 dimensions 2 rows 2 columns
            { {1, 2}, {3, 4} },
            { {5, 6}, {7, 8} }
        };

int[, ,] arr3d3 = new int[2, 2, 3]{    //2 dimensions 2 rows and 3 cols
```

```
        { { 1, 2, 3}, {4, 5, 6} },
        { { 7, 8, 9}, {10, 11, 12} }
    };

arr3d2[0, 0, 0]; // returns
arr3d2[0, 0, 1]; // returns
arr3d2[0, 1, 0]; // returns
arr3d2[0, 1, 1]; // returns
arr3d2[1, 0, 0]; // returns
arr3d2[1, 0, 1]; // returns
arr3d2[1, 1, 0]; // returns
arr3d2[1, 1, 1]; // returns
```

Output?

C# Jagged Arrays: An Array of Array

A jagged array is an array of array. Jagged arrays store arrays instead of literal values.

A jagged array is initialized with two square brackets [][]. The first bracket specifies the size of an array, and the second bracket specifies the dimensions of the array which is going to be stored.

The following example declares jagged arrays.

Example: Jagged Arrays

```
int[][] jArray1 = new int[2][]; // can include two single-dimensional arrays
int[][,] jArray2 = new int[3][,]; // can include three two-dimensional arrays
```

- **Initialization:**

Example: Jagged Array

```
int[][] jArray = new int[2][];

jArray[0] = new int[3]{1, 2, 3};

jArray[1] = new int[4]{4, 5, 6, 7 };
```

You can also initialize a jagged array upon declaration like the below.

Example: Jagged Array

```
int[][] jArray = new int[2][]{
            new int[3]{1, 2, 3},

            new int[4]{4, 5, 6, 7}
        };

jArray[0][0]; //returns 1
jArray[0][1]; //returns 2
jArray[0][2]; //returns 3
jArray[1][0]; //returns 4
jArray[1][1]; //returns 5
jArray[1][2]; //returns 6
jArray[1][3]; //returns 7
```

- ## Accessing Jagged Arrays

You can access a jagged array using two for loops, as shown below.

Example: Jagged Array

```
int[][] jArray = new int[2][]{
            new int[3]{1, 2, 3},

            new int[4]{4, 5, 6, 7}
        };

for(int i=0; i<jArray.Length; i++)
{
            for(int j=0; j < (jArray[i]).Length; j++)
                        Console.WriteLine(jArray[i][j]);
}
```

- ## Two dimensional Jagged Array:

Example: Jagged Array

```
int[][,] jArray = new int[2][,];

jArray[0] = new int[3, 2] { { 1, 2 }, { 3, 4 }, { 5, 6 } };
```

```
jArray[1] = new int[2, 2] { { 7, 8 }, { 9, 10 } };

jArray[0][1, 1]; //returns 4

jArray[1][1, 0]; //returns 9

jArray[1][1, 1]; //returns 10
```

Task:

Student Details Table

Name	Age	Grade	Subjects offered		
Janet	16	9	Math	History	Biology
Claudia	15	9	Music	Geography	Ancient arts
Matt	16	9	Physics	Organic chemistry	Zoology

A student detail table has been given.

1. Identify which array is better for storing data of above table. i.e. use concept of multidimensional and jagged array for implementation.
2. Print data w.r.t arrays that you have implemented in your code.
3. Change grades of student to (9, 10, 7). Those students who are having grades below 9 can drop math, music and physics. Based upon this information what will be array transformation. Print final values.